BIFlow

Des données brutes au tableau de bord vérifié

Rapport technique : pipeline BI multi-agents pour le service public, appliqué aux demandes NYC 311.

Validation technique et métier — novembre 2026
Démarche : Design Thinking (empathie, définition, idéation, prototype, test)
Livrables : code source (dépôt Git), application web de démonstration, présentation, vidéo commentée, ce rapport

Sommaire

1. Résumé 4
2. Démarche : le Design Thinking 5
3. Empathie : utilisateurs et données 6
  3.1 Les personas 6
  3.2 Ce que montrent les données 6
4. Définir : problème et objectifs 7
  4.1 Exigences 7
  4.2 Les cinq preuves du MVP 7
5. Idéer : options et choix techniques 8
6. Prototyper : l'architecture 9
  6.1 L'orchestrateur 9
  6.2 Le socle de données 9
7. Les six agents en détail 10
  7.1 Data Profiler 10
  7.2 Qualité / ETL 10
  7.3 KPI & Sémantique 10
  7.4 Analyste BI 11
  7.5 Dashboard Generator 11
  7.6 Auditeur / XAI 11
8. Sécurité, RGPD et produit 13
  8.1 Mesures de protection 13
  8.2 L'application web 13
  8.3 Les quatre scénarios de démonstration 14
9. Tester : évaluation et itérations 15
  9.1 Méthode 15
  9.2 Résultats 15
  9.3 Test sur données réelles 16
  9.4 Itérations : ce que les tests ont corrigé 17
10. Guide d'utilisation 18
  10.1 Installation 18
  10.2 Commandes principales 18
  10.3 Configuration 18
11. Structure du code source 19
12. Limites et pistes d'amélioration 20
  12.1 Limites assumées 20
  12.2 Prochaines étapes (étape 3 de l'étude) 20
13. Glossaire 21
14. Préparation aux questions 22

1. Résumé

BIFlow automatise une partie du travail d'un analyste BI dans un service public : à partir de données brutes (ici, les demandes de services publics NYC 311), il profile les données, les nettoie, calcule des indicateurs, détecte des tendances et des anomalies, rédige des insights et publie un tableau de bord. Chaque chiffre publié est relié à la requête qui l'a produit et vérifié par un agent d'audit avant publication.

Le principe central est simple : **le LLM comprend et rédige, le code calcule et vérifie**. Le modèle de langage traduit une question en SQL et reformule les textes ; il ne produit jamais un chiffre publié.

Pour la démonstration, une application web React lance le vrai pipeline et montre chaque agent au travail. Quatre scénarios se rejouent en direct : run nominal, panne d'un agent, écart corrigé par l'Auditeur, écart persistant.

Résultats clés

Mesure	Résultat	Objectif
Défauts injectés détectés (profilage)	100 % (9 types)	≥ 95 %
Questions bien traduites en SQL (LLM local, 51 questions)	88 % (baseline sans LLM : 61 %)	≥ 80 %
KPI égaux au calcul manuel	5 sur 5	tous
Anomalie injectée détectée	100 %, 0 faux positif	≥ 80 %
Erreurs volontaires bloquées par l'Auditeur	4 sur 4 (run non publié)	toutes
Insights citant un nombre non justifié	0	0
Run sur données réelles (265 609 demandes, 45 mois)	≈ 10 s, 53 contrôles d'audit sur 53	—
Tests automatisés	95 réussis (dont PostgreSQL et les 4 scénarios de l'application)	—

À retenir. Sur les données réelles, les anomalies détectées correspondent à des événements identifiables (tempête de neige du 24/02/2026, inondations du 29/09/2023), et le BI Analyst développé séparément par l'équipe trouve le même résultat que BIFlow sur le délai médian par borough : deux implémentations indépendantes, même chiffre.

2. Démarche : le Design Thinking

Le projet suit les cinq étapes du Design Thinking. La démarche n'est pas linéaire : la phase de test a renvoyé plusieurs fois au prototype (section 9).

Étape	Question	Ce que nous avons fait	Livrable
1. Empathie	Pour qui, et quelles difficultés ?	Personas ; exploration des données réelles NYC 311	Personas, constats sur les données
2. Définir	Quel problème précis ?	Question « Comment pourrions-nous… » ; exigences ; 5 preuves mesurables	Énoncé du problème, critères de réussite
3. Idéer	Quelles solutions possibles ?	Comparaison d'options (orchestration, LLM, données, analyse)	Étude d'architecture, choix justifiés
4. Prototyper	Comment le construire ?	MVP : 6 agents, LangGraph, DuckDB, application React	Code source, dashboard
5. Tester	Est-ce que ça marche ?	Évaluation sur jeu synthétique, run sur 265 609 demandes réelles	Rapport d'évaluation, corrections

3. Empathie : utilisateurs et données

3.1 Les personas

Quatre profils types, construits à partir de l'analyse du contexte d'un service public qui exploite des demandes citoyennes :

Persona	Besoin	Frustration	Réponse de BIFlow
Analyste BI	Répondre vite aux questions métier	Nettoyage et SQL refaits à chaque fois	Pipeline automatique ; questions en français
Responsable de service	Un chiffre fiable pour décider	Des chiffres qui changent d'un rapport à l'autre	KPI définis une fois dans un catalogue commun
DPO, auditeur	Prouver la conformité et l'origine des chiffres	Impossible de retrouver d'où vient un nombre	Journal des requêtes, empreintes, audit
Citoyen	Un service réactif, des données protégées	Ses données exposées dans un rapport	Minimisation, seuil anti-identification

3.2 Ce que montrent les données

Le jeu NYC 311 (NYC Open Data, jeu `erm2-nwe9`) compte **22 950 021 demandes** publiées entre janvier 2020 et le 27/10/2026, soit environ **324 000 par mois** en 2026. Nous avons travaillé sur un extrait de **265 609 demandes** (janvier 2023 à septembre 2026) issu de l'acquisition bornée réalisée par l'équipe : environ 2 % des demandes de chaque mois et de chaque borough, sélectionnées par suffixe d'identifiant.

Le profilage de cet extrait par notre Data Profiler a trouvé :

- **1 089** demandes au statut « Closed » sans date de clôture ;
- **284** demandes hors des 5 boroughs (« Unspecified » ou vide) ;
- **139** demandes clôturées avant d'avoir été créées (supprimées par l'ETL, gardées en quarantaine).

Trois leçons en découlent : les défauts existent vraiment ; le texte libre et les coordonnées précises sont des données personnelles ; l'extrait est un échantillon systématique **non probabiliste**, donc ses volumes ne s'extrapolent pas à toute la ville (le dashboard l'affiche en réserve).

4. Définir : problème et objectifs

Comment pourrions-nous… permettre à un service public d'obtenir automatiquement des indicateurs fiables, expliqués et conformes au RGPD, à partir de données brutes imparfaites ?

4.1 Exigences

Exigence	Définition	Mécanisme dans BIFlow
Fiabilité	Chaque chiffre vient d'une requête enregistrée et rejouable	Catalogue YAML, journal des requêtes, rejeu par l'Auditeur
Traçabilité	On retrouve la source, la règle et la requête de chaque valeur	Instantané + empreinte SHA-256, `query_ids` dans le dashboard
Confidentialité	Aucune donnée personnelle exposée ni envoyée au LLM	Minimisation, seuil N = 10, contrôle de la question
Autonomie	Questions en langage naturel, sans SQL	Question → SQL par LLM, avec baseline de repli

4.2 Les cinq preuves du MVP

Le MVP est la plus petite version qui prouve l'idée. Les critères de réussite ont été fixés avant les tests :

Preuve	Ce qu'on montre	Réussi si	Objectif
1. Analyser un jeu de données	Profil automatique	Les défauts injectés sont trouvés	≥ 95 %
2. Répondre à une question	Question traduite en SQL	La requête donne le bon résultat	≥ 80 %
3. Calculer un KPI correct	KPI calculé depuis le catalogue	Valeur = calcul manuel	tous
4. Détecter une anomalie	STL + z-score	Bonne zone, bon jour	≥ 80 %
5. Afficher avec traçabilité	Dashboard + journal d'audit	Chaque chiffre relié à sa requête	100 %

5. Idéer : options et choix techniques

Chaque besoin a été confronté à plusieurs options dans l'étude d'architecture. Le critère principal : la fiabilité et l'explicabilité, puisqu'il s'agit de chiffres officiels.

Besoin	Retenu	Pourquoi	Écarté, et pourquoi
Orchestration	LangGraph	Flux déterministe, reprise après erreur, boucle de correction, validation humaine possible	CrewAI, AutoGen : l'outil décide seul de l'ordre des tâches, difficile à justifier
LLM	Mistral (API) ou modèle local (Ollama)	Éditeur européen ; le modèle local garde les données sur la machine	API américaines : données hors de l'administration (référence de qualité seulement)
Stockage	PostgreSQL	Base déjà en place, plusieurs utilisateurs	Entrepôt cloud : hors de l'administration, payant
Exécution SQL	DuckDB (bac à sable)	Rapide, sans serveur, isolable en lecture seule	Exécuter le SQL du LLM sur la base de production
Anomalies	STL + z-score robuste	Simple à expliquer : « X demandes au lieu de Y attendues »	Autoencodeur : boîte noire ; Isolation Forest seulement s'il fait mieux
Dashboard	React + FastAPI	La spécification JSON est générée par un agent ; l'application lance et suit le vrai pipeline (Streamlit au premier prototype)	Power BI, Tableau : outils graphiques, payants, non pilotables

L'idée clé

Le LLM fait…	Le code fait…
comprendre la question en français	calculer chaque KPI à partir du catalogue
proposer une requête SQL	exécuter le SQL en bac à sable et appliquer le seuil
rédiger les insights pour des non-techniciens	vérifier chaque nombre cité dans un texte
—	rejouer chaque requête et décider de publier ou non

Écart assumé avec l'étude : LangGraph est utilisé dès le MVP (au lieu d'un simple script), car le socle était déjà construit et testé ; PostgreSQL stocke les données et DuckDB sert uniquement de bac à sable SQL.

6. Prototyper : l'architecture

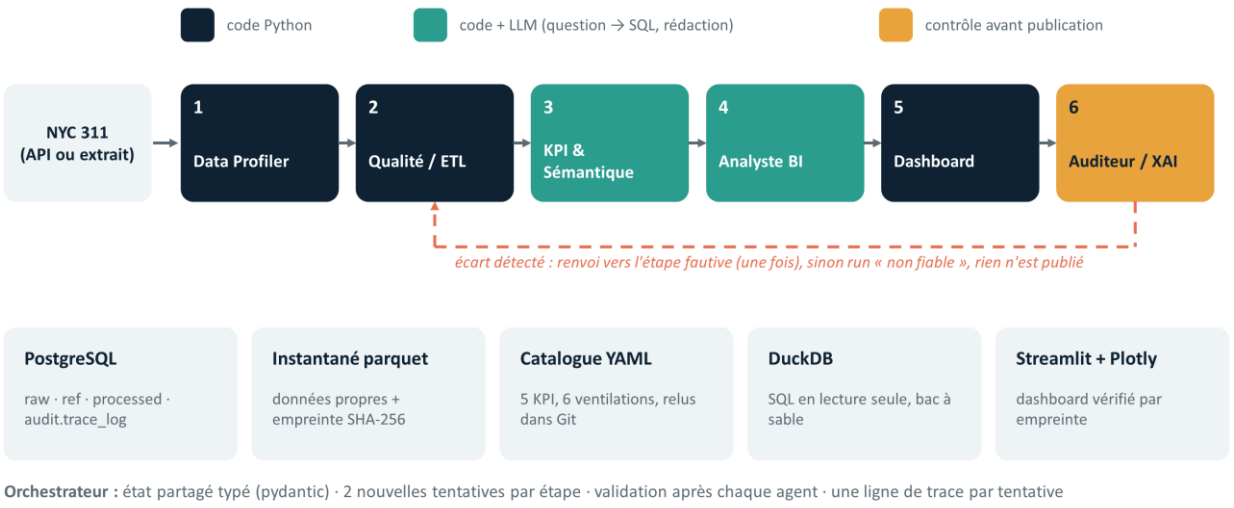


Figure 1 — Le pipeline : 6 agents orchestrés par LangGraph, la boucle de l'Auditeur et le socle technique

6.1 L'orchestrateur

Mécanisme	Comportement
État partagé	`PipelineState` (pydantic) : sorties de chaque agent, validations, erreurs, historique, corrections, constats de l'Auditeur, statut
Nouvelle tentative	Une étape qui lève une exception ou échoue à la validation est relancée 2 fois (`BIFLOW_MAX_RETRIES`)
Échec	Après le dernier essai : run `failed`, les étapes suivantes ne s'exécutent pas, rien n'est publié
Validation	Après chaque agent : schéma pydantic de la sortie + contrôles métier (`validate()`)
Boucle de l'Auditeur	Un constat de sévérité « error » renvoie vers l'étape fautive la plus en amont, une seule fois ; si le problème persiste : run `unreliable`, non publié
Traçabilité	Chaque tentative de chaque étape écrit une ligne dans `audit.trace_log` (PostgreSQL), plus une ligne de synthèse `__run__`
Publication	`run.json` à chaque run ; `published.json` (pointeur lu par le dashboard) seulement si le run réussit

6.2 Le socle de données

Élément	Rôle
`raw.nyc311`	Demandes brutes, clé `unique_key` ; ingestion rejouable (upsert)
`ref.boroughs`, `ref.sla_targets`	Population 2020 (US Census Bureau) ; délais cibles par type de plainte (provisoires)
`processed.nyc311_clean`	Couche propre, écrite par l'agent Qualité / ETL
Instantané parquet	Copie figée des données propres du run, identifiée par une empreinte SHA-256
`kpi_catalog.yaml`	Définitions communes : 5 KPI et 6 ventilations, relues dans Git
Sources possibles	PostgreSQL, ou un extrait parquet/CSV (`--file`), avec sa fiche de provenance `.meta.json`

7. Les six agents en détail

Un agent n'est pas forcément un LLM : c'est un composant avec un rôle, une entrée et une sortie. Chaque agent est une classe de `src/biflow/agents/` avec une méthode `run()` (le travail) et `validate()` (les contrôles).

7.1 Data Profiler

Charge les données brutes et produit un profil : nombre de lignes, taux de valeurs nulles, plage de dates, distributions. Surtout, il liste les **défauts typés**, chacun sur une colonne précise avec son nombre de lignes et quelques identifiants en exemple : doublon, valeur manquante, mauvais type, format invalide (code postal), valeur invalide (borough), hors plage (coordonnées hors de New York), incohérence (clôture avant création). Une colonne facultative absente de l'extrait (code postal, coordonnées, texte de résolution) est signalée, puis ajoutée vide.

7.2 Qualité / ETL

Applique 12 règles, chacune comptée et tracée :

- conversion des types (valeur invalide → NULL) ;
- suppression : identifiant absent ; date de création absente ou invalide ; doublons (dernière version gardée) ; clôture avant création ; type de plainte absent ;
- correction : borough hors des 5 → UNSPECIFIED ; code postal invalide → NULL ; coordonnées hors de New York → NULL ;
- minimisation RGPD : coordonnées arrondies à 2 décimales (environ 1 km) ; texte libre `resolution_description` retiré ;
- enrichissement : délai de résolution, jour, heure, objectif SLA et respect du délai.

Il écrit l'**instantané** parquet et son empreinte, et garde chaque ligne supprimée en **quarantaine** avec la règle qui l'a écartée : aucune ligne ne disparaît sans trace.

7.3 KPI & Sémantique

Calcule les KPI du catalogue par SQL, dans DuckDB, sur l'instantané. Chaque requête est enregistrée (SQL, empreinte de l'instantané, résultat, date).

KPI	Définition	Unité
total_requests	Nombre de demandes après nettoyage	nombre
closure_rate	Demandes clôturées / demandes totales	ratio
median_resolution_hours	Médiane de (clôture − création), demandes clôturées	heures
sla_compliance_rate	% clôturées sous le délai cible (cibles provisoires)	%
requests_per_10k	Demandes pour 10 000 habitants (recensement 2020)	pour 10 000

Six ventilations complètent les KPI (par borough, par type de plainte, par jour…). Tout agrégat portant sur **moins de 10 demandes est masqué** (seuil anti-identification).

Question → SQL

- **Périmètre** : une question de cause (« pourquoi »), de moyens (« combien d'agents recruter ») ou de prévision reçoit un refus motivé, sans requête ni appel au LLM.
- **Contrôle de la question** : e-mail, téléphone, adresse et numéro de demande sont masqués avant l'envoi.
- **Prompt** : schéma des tables, définitions des KPI, modalités des dimensions portées par au moins 10 demandes ; jamais une ligne de données.
- **Bac à sable** : une seule instruction SELECT, tables autorisées, accès aux fichiers coupé, 10 s et 1 000 lignes au maximum ; la requête doit fournir une colonne `n` (effectif) pour appliquer le seuil.
- **Correction** : en cas d'erreur, le message est renvoyé une fois au LLM ; sinon la baseline à règles prend le relais.
- **Contrôle croisé** : si la baseline reconnaît la question, ses règles donnent une seconde réponse ; une divergence est affichée « à vérifier ».

7.4 Analyste BI

Le code calcule les faits ; le LLM ne fait que reformuler.

- **Tendance** : moyenne journalière de la dernière année contre l'année précédente quand l'historique couvre 2 ans (fenêtres de 28 ou 7 jours sinon).
- **Anomalies** : décomposition STL (saisonnalité hebdomadaire, version robuste), puis z-score robuste du résidu (médiane et MAD), avec un plancher de Poisson (l'échelle ne descend pas sous la racine du volume attendu) ; seuil $|z| > 5$; ville entière et chaque borough assez grand.
- **Explication** : pour chaque hausse, le type de plainte qui s'écarte le plus de sa moyenne des 28 jours précédents, présenté comme une piste, jamais comme une cause.
- **Rédaction** : chaque insight porte ses preuves (`evidence`). Si le LLM écrit un nombre absent de ces preuves, son texte est rejeté et le gabarit est conservé.

7.5 Dashboard Generator

Choisit les graphiques par règles (KPI → carte, dimension temporelle → courbe avec anomalies, dimension catégorielle → barres) et écrit la spécification `dashboard_spec.json`, avec, pour chaque élément, les identifiants des requêtes d'origine. L'application Streamlit vérifie l'empreinte de cette spécification avant de l'afficher.

7.6 Auditeur / XAI

C'est le code qui vérifie, pas le LLM. Contrôles effectués à chaque run (53 sur les données réelles) :

- empreinte de l'instantané recalculée = empreinte enregistrée ;
- lignes profilées = lignes lues par l'ETL ;
- KPI affiché = résultat enregistré de sa requête ;
- reproductibilité : chaque requête rejouée redonne le même résultat ;
- recalcul indépendant des principaux KPI en pandas, sans SQL ;
- aucun agrégat sous le seuil ; aucun nombre non justifié dans les insights ;
- empreinte et valeurs du dashboard = KPI vérifiés.

Chaque écart devient un constat adressé à l'étape fautive. L'Auditeur produit aussi une explication en clair de la façon dont les chiffres ont été obtenus.

8. Sécurité, RGPD et produit

8.1 Mesures de protection

Risque	Mesure	Où dans le code
SQL dangereux généré par le LLM	Bac à sable DuckDB : copie en mémoire, SELECT seul, tables autorisées, fichiers coupés, configuration verrouillée, temps et lignes limités	`semantic/sandbox.py`
Identification d'un foyer	Tout résultat sur moins de 10 demandes est masqué (N à fixer avec le DPO)	`agents/kpi_semantic.py`
Données personnelles dans les données	Texte libre retiré, coordonnées arrondies à environ 1 km	`agents/quality_etl.py`
Données personnelles dans la question	E-mail, téléphone, adresse, n° de demande masqués avant envoi	`llm/privacy.py`
Chiffre inventé par le LLM	Tout nombre d'un texte doit exister dans les preuves calculées	`analysis/numbers.py`
Réponse trompeuse	Questions hors périmètre refusées avec explication ; contrôle croisé LLM / baseline	`semantic/scope.py`
Données envoyées hors de la machine	Modèle local possible (Ollama) ; jamais de ligne brute envoyée au LLM	`llm/client.py`
Chiffre modifié après l'audit	Le dashboard refuse une spécification dont l'empreinte a changé	`dashboard/app.py`, `api/app.py`

8.2 L'application web

Une application React, servie par une API FastAPI (src/biflow/api) qui lance le vrai pipeline LangGraph. Cinq pages : Scénarios (lancer un run et suivre chaque agent en direct, sur une ligne à six stations), Tableau de bord (KPI, insights, graphiques, « Voir le SQL » sous chaque chiffre ; seul un run approuvé par l'Auditeur est affiché, après vérification de l'empreinte de sa spécification), Poser une question (question en français sur l'instantané audité, avec les mêmes garde-fous), Audit et provenance, et Historique des runs. Lancement : .\scripts\tasks.ps1 web, puis http://localhost:8000.



Figure 2 — Le tableau de bord publié (données réelles) : 5 KPI, chacun relié à sa requête

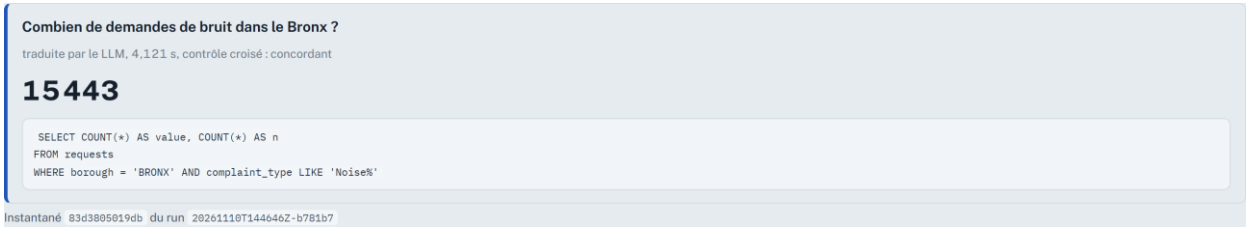


Figure 4 — « Poser une question » : SQL écrit par le LLM local, contrôle croisé concordant

8.3 Les quatre scénarios de démonstration

Chaque scénario lance le vrai pipeline sur l'extrait choisi ; les pannes et les écarts sont injectés avec les mêmes options que la ligne de commande (--simulate-failure, --simulate-audit-flag). Les quatre scénarios sont aussi des tests automatiques (tests/test_api.py).

Scénario	Ce qui se passe	Résultat
Run nominal	Les 6 agents s'enchaînent ; l'Auditeur valide 53 contrôles sur 53.	Publié
Panne d'un agent	L'agent choisi échoue à chaque essai : 3 essais, puis l'orchestrateur arrête le run ; les agents suivants ne tournent pas.	Échec, rien n'est publié
Écart corrigé par l'Auditeur	L'Auditeur signale un écart sur une étape : le run repart de cette étape, le second audit passe.	Publié
Écart persistant	L'écart est toujours là après le renvoi.	Non fiable ; le tableau de bord garde le dernier run validé

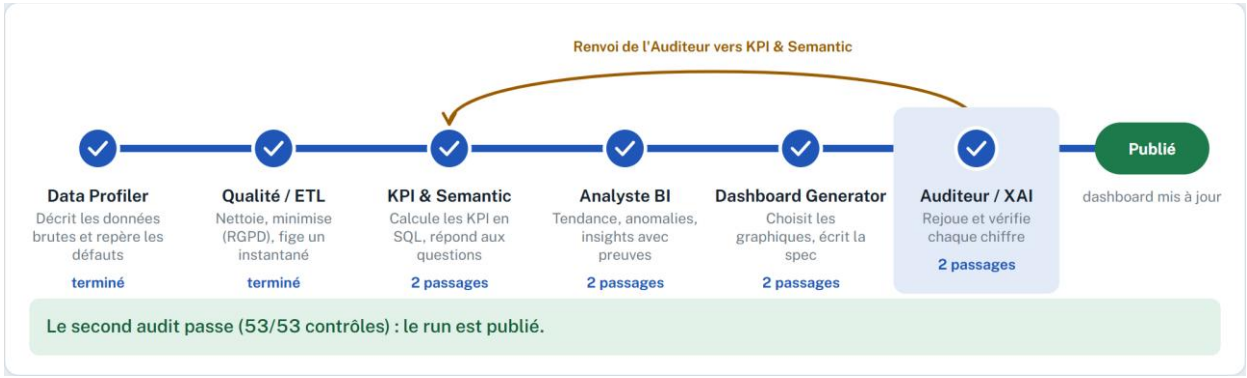


Figure 5 — Scénario « écart corrigé » : l'Auditeur renvoie le run vers KPI & Semantic, le second audit passe

9. Tester : évaluation et itérations

9.1 Méthode

`python -m biflow.evaluation` génère un jeu synthétique de type NYC 311 dont tout est connu à l'avance : 8 semaines (à partir du 05/01/2026), 5 boroughs, 7 types de plainte, une saisonnalité hebdomadaire, un pic injecté (bruit résidentiel multiplié par 3 à Brooklyn un jour donné) et 9 défauts injectés sur des lignes distinctes.

Colonne	Défaut injecté	Lignes
created_date	mauvais type (« pas une date »)	10
closed_date	incohérence (clôture 2 h avant la création)	12
borough	valeur manquante / valeur invalide	20 / 6
latitude	mauvais type / hors plage	10 / 8
longitude	hors plage	8
incident_zip	format invalide	10
unique_key	doublon	15

La génération de SQL est mesurée sur **51 questions** : 30 « catalogue » (formulations prévues par les règles de la baseline) et 21 reformulations libres, plus 3 questions hors périmètre. Une requête est correcte si elle donne le même résultat que la réponse attendue, calculée en pandas, même si le SQL est écrit différemment. Quatre erreurs volontaires testent l'Auditeur : instantané modifié après son empreinte, KPI faussé, insight citant un chiffre inventé, carte du dashboard modifiée.

9.2 Résultats

Critère	Baseline sans LLM	LLM local (qwen2.5:7b)	Objectif
Profilage	100 %	100 %	≥ 95 %
SQL, 51 questions	61 %	88 %	≥ 80 %
… 30 questions « catalogue »	100 %	97 %	—
… 21 reformulations libres	5 %	76 %	—
KPI = calcul manuel	5 / 5	5 / 5	tous
Anomalie (bonne zone, bon jour)	100 %, 0 faux positif	idem	≥ 80 %
Insights avec un nombre non justifié	0	0	0
Erreurs volontaires détectées	4 / 4	4 / 4	toutes
Questions hors périmètre refusées	3 / 3, 0 refus à tort	3 / 3	—
Temps par question	≈ 0,004 s	≈ 2,2 s	mesurer

Contrôle croisé : quand le LLM et la baseline concordent, la réponse du LLM était juste dans 25 cas sur 25 ; quand ils divergent, dans 15 cas sur 19. Une divergence signale donc une réponse à vérifier.

Réserve. Une première mesure avec le LLM donnait 76 % au total. Des règles générales ont ensuite été ajoutées au prompt (une moyenne s'écrit AVG, un total tient en une ligne, les modalités des dimensions sont fournies), puis la mesure a été relancée : 88 %. Ces règles ayant été écrites après avoir vu les erreurs sur ces mêmes questions, le résultat est légèrement optimiste : il faut le confirmer sur un jeu de questions neuf, et avec Mistral.

9.3 Test sur données réelles

Sur les 265 609 demandes réelles : run complet en environ 10 secondes, 139 lignes supprimées (clôture avant création), 53 contrôles d'audit réussis sur 53. KPI : 97,7 % de demandes clôturées, délai médian de 7,3 h, Staten Island le plus lent (15,7 h), Queens le plus rapide (4,6 h).

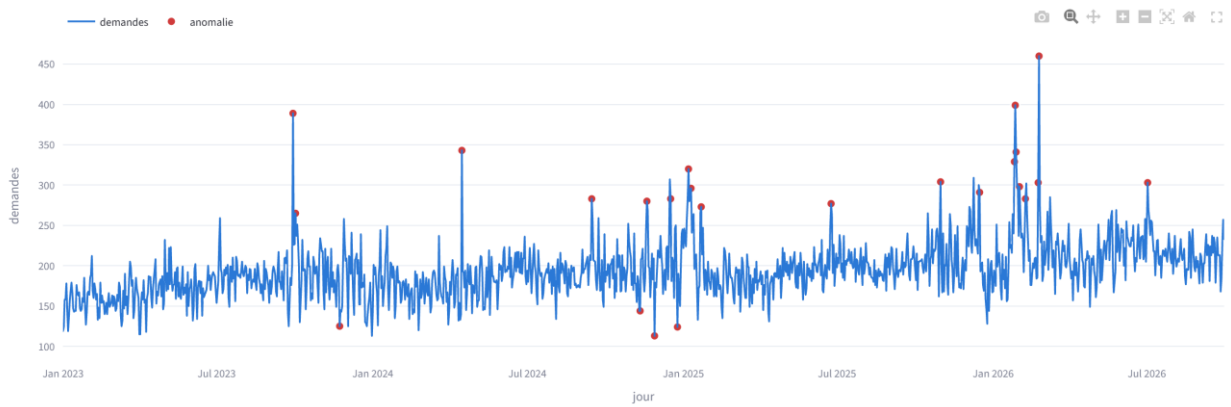


Figure 3 — Demandes par jour, 2023-2026 : les anomalies détectées sont en rouge

Date, zone	Observé / attendu	Type le plus en hausse	Lecture
24/02/2026, toute la ville	460 / ≈ 218	Snow or Ice (231 contre 20,5 par jour)	tempête de neige
29/09/2023, toute la ville	389 / ≈ 182	Sewer Maintenance	inondations à New York
15/09/2024, Bronx	139 / ≈ 36	Noise - Residential (108 contre 6,3)	à investiguer

Contrôle croisé avec l'équipe : le BI Analyst développé séparément (dépôt `Bi_Flow_Project`) trouve sur la même extraction STATEN ISLAND comme borough au délai médian le plus long, 0,655677 jour = 15,736 h (n = 9 874). BIFlow trouve 15,736 h (n = 9 876) ; l'écart de 2 demandes vient des règles de nettoyage.

9.4 Itérations : ce que les tests ont corrigé

Défaut trouvé	Symptôme	Correction
Ordre des résultats instable	L'Auditeur signalait des runs « non fiables » au hasard (égalités dans un ORDER BY)	Tri secondaire déterministe dans chaque requête du catalogue
Fausses alertes d'anomalie	Du bruit normal détecté comme anomalie	Lissages STL plus longs, plancher de Poisson, seuil z = 5
Limite de 1 000 lignes	La série journalière réelle (1 369 jours) bloquait le run	Limite propre au SQL du catalogue (100 000), relu dans Git
Tendance sans sens	« Première semaine contre dernière » sur 45 mois	Comparaison année contre année précédente
Réponse à côté	« Quelle agence... ? » renvoyait le total	La baseline répond « non reconnue » ; contrôle croisé avec le LLM
Script Windows bloqué	Les journaux Python (stderr) arrêtaient PowerShell 5.1	Code de sortie testé au lieu de l'arrêt sur erreur

10. Guide d'utilisation

10.1 Installation

```
python -m venv .venv
.\.venv\Scripts\python -m pip install -e ".[dev]"
Copy-Item .env.example .env          # puis renseigner POSTGRES_PASSWORD
docker compose up -d --wait          # PostgreSQL 16 + Adminer (optionnel)
.\.venv\Scripts\python -m biflow.db.init_db
ollama pull qwen2.5:7b               # optionnel : LLM local
```

10.2 Commandes principales

Commande	Effet
`.\scripts\tasks.ps1 web`	Application web (http://localhost:8000) : les 4 scénarios en direct, tableau de bord, questions. LLM local : BIFLOW_LLM_PROVIDER=ollama
`.\scripts\demo.ps1`	Run sur l'extrait 2023-2026 avec 4 questions, puis dashboard (http://localhost:8501)
`.\scripts\demo.ps1 -Llm ollama`	Même démonstration avec le LLM local
`python -m biflow.run --no-db --file <extrait>`	Pipeline complet sur un fichier, sans PostgreSQL
`python -m biflow.run --question "..."`	Ajoute une question métier (répétable)
`python -m biflow.run --simulate-audit-flag kpi_semantic --persistent`	Démonstration : run non fiable, rien n'est publié
`python -m biflow.evaluation`	Évaluation des 5 preuves (rapport dans `data/processed/evaluation/`)
`python -m biflow.ingestion.import_pages <dossier> --name <nom>`	Importe des pages brutes en un extrait avec fiche de provenance
`python -m pytest`	95 tests (les tests PostgreSQL sont ignorés sans base)

10.3 Configuration

Variable	Rôle	Défaut
`BIFLOW_LLM_PROVIDER`	auto (Mistral si clé, sinon aucun), mistral, ollama, none	auto
`MISTRAL_API_KEY`, `MISTRAL_MODEL`	API Mistral	vide, mistral-small-latest
`OLLAMA_MODEL`, `OLLAMA_BASE_URL`	Modèle local	qwen2.5:7b, http://localhost:11434/v1
`BIFLOW_MIN_GROUP_SIZE`	Seuil anti-identification	10
`BIFLOW_SQL_MAX_ROWS`, `BIFLOW_SQL_TIMEOUT_SECONDS`	Limites du SQL généré	1000, 10
`BIFLOW_MAX_RETRIES`, `BIFLOW_MAX_FEEDBACK_ROUNDS`	Nouvelles tentatives, tours de l'Auditeur	2, 1

11. Structure du code source

```
biflow/
├── docker-compose.yml          PostgreSQL 16 + Adminer
├── scripts/demo.ps1, tasks.ps1  démonstration et raccourcis Windows
├── frontend/                   application web React (Vite + TypeScript)
├── docs/DEMO.md, SUMMARY.md     déroulé de démonstration, synthèse
├── src/biflow/
│   ├── run.py                 CLI du pipeline
│   ├── orchestrator/          state, graph (LangGraph), validation, tracing, report
│   ├── agents/                base.py + un fichier par agent
│   ├── semantic/              kpi_catalog.yaml, sandbox.py, text_to_sql.py, scope.py
│   ├── analysis/              anomalies.py (STL + z-score), numbers.py (nombres cités)
│   ├── llm/                   client.py (Mistral / Ollama), privacy.py
│   ├── dashboard/app.py       Streamlit + Plotly (premier prototype)
│   ├── api/                   FastAPI : scénarios en direct, tableau de bord publié
│   ├── evaluation/            jeu synthétique + python -m biflow.evaluation
│   ├── ingestion/             API Socrata, import de pages
│   ├── db/                    engine, models, init_db, loader
│   └── common/                config, schéma, données, référentiels, instantanés
└── tests/                     95 tests pytest
```

Contrat d'un agent : `name` (clé dans l'état), `output_model` (modèle pydantic de sortie), `run(state)` qui retourne la sortie sans modifier l'état, et `validate(output, state)` qui retourne des contrôles. En cas d'erreur, l'agent lève une exception : l'orchestrateur relance puis marque le run en échec.

12. Limites et pistes d'amélioration

12.1 Limites assumées

- **Mistral non testé** faute de clé : l'apport du LLM a été mesuré avec un modèle local open-weight.
- **Mesure du LLM légèrement optimiste** (prompt ajusté sur le même jeu de questions) : à confirmer sur un jeu neuf.
- **Échantillon non probabiliste** : les volumes et les taux pour 10 000 habitants ne valent pas pour toute la ville.
- **Délais cibles SLA provisoires**, à valider avec le métier.
- **Détection des données personnelles par motifs** : un nom propre isolé n'est pas détecté.
- **Biais des périodes récentes** : seules les demandes déjà clôturées entrent dans les délais.
- **Un seul jeu de données** testé (NYC 311).

12.2 Prochaines étapes (étape 3 de l'étude)

Ajout	Type	Quand
Validation humaine des nouvelles règles et KPI (pause LangGraph)	Obligatoire	Dès que le MVP fonctionne
Contrôle d'accès par rôle	Obligatoire	Avant les données réelles
Hébergement conforme du LLM, AIPD	Obligatoire	Avant les données réelles
Comparaison Mistral / modèle local sur 50 questions neuves	Mesure	Prochaine itération
Isolation Forest + SHAP + MLflow	Optionnel	S'ils détectent mieux que STL
dbt, Pandera, Langfuse	Optionnel	Quand les règles se multiplient
API FastAPI, déploiement Docker	Optionnel	Mise en production

13. Glossaire

Terme	Définition
Agent	Composant avec un rôle, une entrée et une sortie ; il n'utilise un LLM que s'il doit comprendre ou rédiger du texte
Baseline	Version de référence sans LLM (règles par mots-clés), qui sert aussi de repli
Instantané	Copie figée des données propres d'un run, identifiée par son empreinte
Empreinte (SHA-256)	Résumé cryptographique d'un contenu : la moindre modification la change
Couche sémantique	Définitions communes des KPI (catalogue YAML), utilisées par tous les agents
Seuil anti-identification	Nombre minimal de demandes (10) sous lequel un résultat est masqué
STL	Seasonal-Trend decomposition using Loess : sépare tendance, saisonnalité et résidu
z-score robuste	(valeur − médiane) / (1,4826 × MAD) ; MAD = écart absolu médian, peu sensible aux valeurs extrêmes
Plancher de Poisson	Pour des comptages, l'écart-type minimal est la racine du volume attendu
Bac à sable	Environnement isolé où le SQL ne peut ni modifier ni lire autre chose que les tables autorisées
Contrôle croisé	Comparaison des réponses du LLM et de la baseline à une même question
RGPD, AIPD, DPO	Règlement européen sur les données personnelles ; analyse d'impact ; délégué à la protection des données

14. Préparation aux questions

Pourquoi ne pas utiliser simplement ChatGPT sur les données ?
Il ne garantit ni la fiabilité (chiffres inventés possibles), ni la confidentialité (lignes brutes envoyées hors de l'administration), ni la traçabilité. Dans BIFlow, le LLM ne voit jamais une ligne de données et ne produit aucun chiffre.

Le LLM peut-il inventer un chiffre ?
Non dans ce qui est publié : les KPI viennent du SQL du catalogue ; les insights sont rédigés à partir des preuves calculées, et tout nombre absent de ces preuves fait rejeter le texte (le gabarit est gardé). L'évaluation le vérifie : 0 nombre non justifié.

Et si le SQL du LLM est faux ?
S'il est dangereux ou invalide, le bac à sable le refuse et l'erreur est renvoyée une fois au LLM, puis la baseline prend le relais. S'il est valide mais faux, le contrôle croisé avec la baseline signale la divergence (« à vérifier »). C'est la limite principale : 88 % de réponses justes, pas 100 %.

Pourquoi LangGraph plutôt que CrewAI ou AutoGen ?
Pour des chiffres officiels, l'ordre des étapes doit être fixe et justifiable. LangGraph donne un graphe déterministe avec reprise après erreur et boucle de correction ; CrewAI et AutoGen laissent les agents décider de l'ordre.

Pourquoi DuckDB en plus de PostgreSQL ?
PostgreSQL stocke. DuckDB exécute le SQL du LLM sur une copie en mémoire de l'instantané, sans accès aux fichiers ni à la base : rien ne peut être modifié ou lu en dehors des tables autorisées.

Comment détectez-vous une anomalie ?
STL sépare la tendance et la saisonnalité hebdomadaire ; on calcule le z-score robuste du résidu ; un jour est anormal si $|z| > 5$. Le seuil et le plancher de Poisson ont été réglés pour éviter les fausses alertes (0 faux positif sur le jeu de test).

Comment savez-vous que l'Auditeur fonctionne ?
On l'a saboté volontairement 4 fois (instantané, KPI, insight, dashboard) : il a détecté les 4 écarts et le run n'a pas été publié. Il a aussi trouvé un vrai bug (ordre non déterministe des résultats).

Pourquoi un jeu synthétique ?
Sur des données réelles, on ne connaît pas la vérité. Avec un jeu synthétique, on sait exactement quels défauts et quelle anomalie on a injectés, et on calcule les KPI à la main pour comparer.

Vos données représentent-elles New York ?
Non pour les volumes : l'extrait est un échantillon systématique d'environ 2 % par mois et par borough, non probabiliste. Les comparaisons entre boroughs et les délais restent informatifs ; le dashboard affiche cette réserve.

Quelle est la valeur pour un client ?
Moins de temps passé à nettoyer et à écrire du SQL, des chiffres dont on peut prouver l'origine, et la conformité RGPD dès la conception. Le pipeline s'adapte à toute organisation qui reçoit des demandes avec un type, un lieu et des dates.